PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

Fuchao Sheng, Yongjian Xie, Yongming Li, Sanjiang Li *

***Abstract*—A distributed quantum computer runs one circuit across several small cores joined by links that distribute entanglement on demand. Under the usual cost model the compiler's objective is to spend as few Einstein–Podolsky–Rosen (EPR) pairs as possible on gates that straddle cores. Existing routers choose among inter-core actions while simultaneously staging them, so an action's apparent value depends on the layout the compiler happens to occupy. We present PORTER, a compiler that instead scores a complete inter-core *plan* — which primitive fires, which gates it resolves, and which readiness conditions it needs — before any preparation SWAP for it exists, commits to one, and only then prepares it. PORTER supports all three known inter-core primitives, including cat-entangler packets that amortise one pair over a run of remote gates sharing a control, and adds capacity-aware relief, a deterministic initial layout, and a per-circuit choice of teleport-lookahead horizon. On MQT-Bench suites at 25, 64 and 80 logical qubits PORTER uses 59.6%, 41.6% and 68.2% fewer EPR pairs in geometric mean than published TELESABRE at three routes each, and completes all twenty-four instances where TELESABRE completes seventeen. Against DSABRE it is competitive at the sizes both target — $1.02\times$ its pooled EPR at $0.91\times$ its compile time under each compiler's own protocol — and pulls ahead as the machine grows, using $0.72\times$ its EPR pairs on a 100–360-qubit scale-up series. Controlled ablations measure what each mechanism contributes. The implementation, harness and raw results are public.**

***Index Terms*—distributed quantum computing, quantum circuit compilation, multi-core quantum architectures, gate teleportation, cat-entangler packets, plan-oriented routing, entanglement cost**

I. INTRODUCTION

A distributed quantum computer (DQC) runs one circuit across several small cores joined by links that distribute entanglement on demand [1], [2]. Any two-qubit gate whose operands sit in different cores must be paid for with Einstein–Podolsky–Rosen (EPR) pairs, and distributing a pair is far slower and less reliable than any operation inside a core [3], [4]. Pairs consumed is therefore the metric we minimise (Section II-B).

A pair can be spent three ways. Teleporting a *state* (TELEDATA) relocates a logical qubit, making every later gate it meets in its new core local [5]. Teleporting a *gate* (TELEGATE) executes one straddling controlled gate through a cat-entangler, with neither operand moving [6], [7]. Holding that cat-entangled resource open across a run of straddling gates that share a control costs one pair for the whole run — a *packet*, following AutoComm's gate-sequence amortisation [8]. The last two work for any controlled two-qubit gate; our benchmarks use CX throughout (Section II-A). Which primitive is cheapest depends on the circuit structure where it is applied, so none dominates.

The choice must be made before its price is known. Each alternative becomes executable only after its own preparation: the right operand beside the right communication port, that port free, the destination holding a spare slot. A compiler performs that work only for the alternative it selects. Scoring each action together with the preparation it currently needs therefore confuses two things. It ranks candidates by the layout the compiler happens to occupy, not by the progress they buy. And staging a candidate that then loses still perturbs the layout.

PORTER — a *Plan-Oriented Router for Teleportation-Enabled aRchitectures* — separates the two questions. Its unit of choice is a *plan*: a complete specification of one inter-core action and the conditions it needs, omitting the SWAP sequence that would establish them. Plans are compared on the gate progress they buy, one is committed, and only then are preparation SWAPs generated for it. Around this loop PORTER adds two-tier capacity relief, which manufactures free slots when every candidate is blocked, and a single deterministic initial layout in place of the best-of-N search its predecessors run.

Results. We evaluate on MQT-Bench [9] suites at 25, 64 and 80 logical qubits, on a 100–360-qubit scale-up series, and on two networks of IBM heavy-hex cores — forty-eight circuit–architecture instances. Against TELESABRE [10], the strongest published router we identified under this cost model, PORTER uses substantially fewer EPR pairs: over the instances both complete it lowers geometric-mean consumption by 59.6%, 41.6% and 68.2% on the three suites at three routes each, and it completes all twenty-four, including the seven — among them an 80-qubit, 23,381-CX circuit — on which TELESABRE does not converge.

The second comparison is with DSABRE [11], concurrent work on the same problem and the closest point of reference we have. PORTER is competitive with it at the sizes both target, spending $1.02\times$ its pooled geometric-mean EPR for $0.91\times$ its compile time under each compiler's own protocol, and it pulls ahead as the machine grows: on the 100–360-qubit series it uses $0.72\times$ DSABRE's EPR pairs and wins every instance. Reaching parity with a concurrent compiler from a different design, and then improving on it where the core count rises, is the result we take from this comparison; Section V gives every operating point behind it. The two are close relatives and share four design decisions, set out

F. Sheng, Y. Xie, and Y. Li are with the School of Mathematics and Statistics, Shaanxi Normal University, Xi'an, China; S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).

*ORCID: 0000-0002-3332-2546.

in Section VII so neither is credited with the other's.

Problem. Given a circuit DAG on n logical qubits, a machine of K cores with intra-core coupling graphs, inter-core links and designated communication ports, and an initial layout, emit a legal operation stream realising every gate while minimising the number of EPR pairs consumed. Section II-B states the cost model this objective assumes and what it excludes.

Contributions. We make three.

(i) *The* PORTER *compiler*: a plan-oriented remote stage that scores a complete inter-core action before any preparation SWAP for it exists and commits before preparing (Section III-D), a capacity discipline that treats free slots as a legality condition and manufactures them by a two-tier relief search (Section III-G), and a deterministic initial layout replacing the best-of-N search its predecessors run (Section III-A).

(ii) *Per-circuit selection of the lookahead horizon.* Every SABRE-descended router fixes this parameter by constant, and no tested constant is best across our benchmark — the best value spans a factor of thirty-two. PORTER instead measures how dense the circuit's interaction graph is, uses that to prune a three-value candidate set, routes each and keeps the lowest-EPR result. The configured horizon always competes and wins ties, so the selection cannot return a worse route than routing once. It is what carries PORTER past DSABRE on the scale-up series (Section IV-E1).

(iii) *A controlled study over forty-eight instances*, under native, shared-layout and cross-layout protocols, with ablations that measure what each mechanism contributes — including a negative finding, which Section VI interprets. Every route is certified by a separately implemented replay verifier, and the harness reproducing every table is released.

Section IV-A states the four questions the evaluation answers. Each mechanism is tied to the experiment that assesses it: capacity relief to the completion ablation, the initial layout to the shared- and cross-layout controls, horizon selection to the scale-up series and the star-Multiplier livelock, packets to the action-vocabulary study, and the verifier to fault injection.

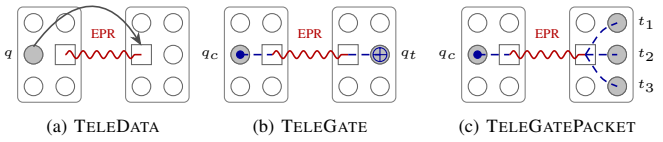
The PORTER implementation, the replay verifier, the full cross-router comparison harness, and the raw per-circuit results are publicly available at https://github.com/ebony72/porter.

II. Model, Primitives, and Problem Formulation

A. Quantum Circuits and Routing

A circuit acts on n logical qubits. We work in the basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ of MQT-Bench's Qiskit-opt3-transpiled circuits [9]: arbitrary one-qubit gates plus the two-qubit CX. A circuit is a directed acyclic graph, with a dependency edge between consecutive gates on each logical qubit. The *front* is the set of gates with no unexecuted predecessor; only front gates can execute or be routed toward execution.

Physical qubits are partitioned into *cores*. Each core has an intra-core coupling graph, and pairs of cores may be joined by an inter-core link whose endpoints are *communication ports*. A *layout* maps every logical qubit to a physical position. A two-qubit gate is *local* if both operands share a core and *remote* otherwise: a local gate that is not yet adjacent needs intra-core SWAPs, a remote gate needs an inter-core primitive. We write $\kappa_\ell(q)$ for the core holding q under layout ℓ, $d_{\mathrm{L}}(c, p, p')$ for intra-core distance and $d_{\mathrm{C}}(c, c')$ for core-graph distance.



(a) TeleData (b) TeleGate (c) TeleGatePacket

Fig. 1. The three communication primitives; each consumes one EPR pair between the two communication ports (squares). (a) TeleData moves qubit q (which must sit adjacent to its free port) onto the free destination port. (b) TeleGate executes one remote CX through a cat-entangler [7] (Appendix E): the target-side port acts as a proxy for the control q_c; neither qubit moves. (c) TeleGatePacket brackets an ordered run of remote CX gates sharing the control q_c between one cat-entangler/disentangler pair, amortising a single EPR pair over the whole run; each target is routed next to the port in turn while the ports stay reserved.

B. Objective, Scope, and Assumptions

Objective. Given a circuit DAG C on n logical qubits, a machine $\mathcal{A}$ of K cores with intra-core coupling graphs, inter-core links and designated communication ports, and an initial layout ℓ_0, produce a legal operation stream that realises every gate of C while minimising the number of EPR pairs consumed. Intra-core SWAP count, circuit depth and compile time are recorded but are not part of the objective and are never traded against EPR pairs by any rule in this paper.

Cost model. One EPR pair is consumed per TeleData, per TeleGate, and per TeleGatePacket — one per packet, however many remote CX gates the packet amortises — and a pair is charged the same regardless of which link carries it. A multi-hop relocation is a sequence of single-hop TeleData actions and is charged once per hop. This is the model TeleSABRE [10] and DSABRE [11] use, which is what makes the three compilers comparable in a single unit.

Communication ports are ordinary physical qubits, occupied for the duration of an action and free otherwise; no core is assumed to have dedicated communication memory. What the model leaves out — link contention, entanglement latency and fidelity, purification, classical communication cost — and what that means for the conclusions drawn here is set out in Section VI-A.

C. Communication Primitives

Three inter-core primitives are in play, sketched in Figure 1; each consumes exactly one EPR pair.

State teleportation (TeleData). Consumes one EPR pair to move a logical qubit from a communication port in one core to a free port in another, after which every later gate that qubit meets in its new core is local. Its value is therefore *latent*: it is paid once and repaid over however many future gates the relocation makes local.

Gate teleportation (TeleGate). Executes a single remote CX with neither operand moving. One EPR pair entangles

the two communication ports; a *cat-entangler* — $\mathrm{CX}(q_c, e_A)$, measurement of e_A, and a classically controlled X on e_B — leaves (q_c, e_B) in the cat state $\alpha|00\rangle + \beta|11\rangle$, so every CX controlled by the remote port e_B acts as if controlled by q_c. A *cat-disentangler* — Hadamard on e_B, measurement, a classically controlled Z on q_c — restores q_c and frees both ports (Appendix E). Its value is entirely *direct*: exactly one gate, and nothing is left behind.

Same-control packets (TELEGATEPACKET). When several remote CX gates share the same control and their targets lie in one core, the run can be bracketed between a *single* cat-entangler and disentangler: the ports stay reserved, each target is routed beside e_B in turn, and the whole run costs one EPR pair instead of one each. This is the amortisation idea of AutoComm [8], specialised to the shared-control case and applied inside a routing loop rather than as a pre-pass.

The three differ in one respect that matters for scoring: how many gates one pair is spread over. TELEDATA amortises over an unknown future, TELEGATE over exactly one gate, TELEGATEPACKET over a run whose length the compiler must discover. Section III-E makes that a direct/latent/lookahead decomposition.

III. THE PORTER COMPILER

Given a circuit and an initial layout ℓ_0 (which PORTER builds itself, Section III-A), PORTER repeats the following loop (Algorithm 1) until every gate has been emitted or a configurable iteration budget is exhausted. Algorithm 1 charts one iteration; lines 11–19 are the plan-first remote stage that distinguishes PORTER from compilers deciding at single-action granularity, and Appendix F walks through them on a concrete front where they change the outcome.

A. Initial Layout

PORTER routes from any initial layout, and the shared- and cross-layout experiments of Section V-B run it on layouts built by other compilers. Left to itself it builds ℓ_0 with one deterministic heuristic — no external partitioner, no best-of-N search — giving every core at least $f_{\min}$=2 free positions.

That floor is a natural requirement rather than a tuned parameter, and DSABRE [11] independently arrives at the same value as its own routing legality condition. A core with no free physical qubit is dead: it cannot vacate an occupied communication port by local SWAPs, so no inter-core action can reach it. A core with exactly one free position is alive but cannot accept a TELEDATA, since the arriving qubit would fill it. Two is therefore the smallest reserve under which every core can both receive a teleport and remain able to make room afterwards. $f_{\min}$ accordingly plays three roles, which we keep apart: this *initial* condition; the *admission threshold* a destination must meet *before* a normal TELEDATA (Section III-D), so at least one position remains after one; and the threshold defining a *reservoir* for capacity relief (Section III-G). It is not maintained after every move: a core admitted at exactly $f_{\min}$ drops to one free, and cores do reach one free position during routing.

Algorithm 1 PORTER `compile`(C, ℓ_0)

1: $\ell \leftarrow \ell_0$; initialise DAG active/frontier sets from C
2: **while** DAG has active gates **do**
3: $F \leftarrow$ DAG front (bounded by `dag_front_limit`)
4: classify F into singles, direct-local CX, local-CX-needing-SWAP, adjacent-remote CX, non-adjacent-remote CX *(Sec. III-B)*
5: **if** singles or direct-local CX present **then**
6: execute them all; remove from DAG; **continue**
7: **end if**
8: **if** local-CX-needing-SWAP present **then**
9: generate candidate intra-core SWAPs, apply the best-scoring one (Eq. 1); **continue** *(Sec. III-C)*
10: **end if**
11: generate normal plans $\mathcal{P}$, blocked plans $\mathcal{B}$ *(Sec. III-D)*
12: score every plan in $\mathcal{P} \cup \mathcal{B}$ *(Sec. III-E)*
13: $\mathsf{p}^* \leftarrow$ first plan of $\mathcal{P}$, in selection-key order, that is ready or admits an improving preparation SWAP
14: **if** p^* is `None` **then**
15: $\mathsf{p}^* \leftarrow$ best tail-relief plan seeded from $\mathcal{B}$ *(Sec. III-G)*
16: **else if** $\mathcal{B} \neq \emptyset$ and $\max_{\mathsf{b}\in\mathcal{B}} \mathrm{total}(\mathsf{b}) - \mathrm{total}(\mathsf{p}^*) > m_{\mathrm{assist}}$ **then**
17: $\mathsf{b}^* \leftarrow \arg\max_{\mathsf{b}\in\mathcal{B}} \mathrm{total}(\mathsf{b})$
18: $\mathsf{p}^* \leftarrow$ best relief plan for b^*'s blocked core *(Sec. III-G)*
19: **end if**
20: **if** p^* is `None` **then**
21: **report deadlock; abort**
22: **end if**
23: apply p^*'s own preparation SWAPs until ready, then execute it
24: **end while**
25: **return** emitted operations, final layout, stats

Temporal locality-greedy. Three steps. *(i) Weighting*: each CX gate $g = (q_i, q_j)$ contributes $\gamma^{\mathrm{asap}(g)}$ to the interaction weight $w(q_i, q_j)$, where $\mathrm{asap}(g)$ is g's as-soon-as-possible layer in the CX-only DAG and γ=0.99, so early interactions weigh more than late ones while repeated interactions accumulate. *(ii) Core assignment*: logical qubits are visited in the order they first appear as a CX operand and each is placed in the core holding its heaviest already-placed neighbour when that core has headroom above $f_{\min}$, falling back to the core with the most remaining reserved capacity. *(iii) Physical placement*: within each core, logicals are laid onto positions by descending interaction weight onto ascending intra-core centrality, so the most-connected qubits take the most central slots.

The steps are complementary. The discount co-locates the pairs that must interact soonest; first-appearance order handles chain-structured circuits, where a balanced-first-layer rule would scatter the chain across cores; and the centrality step keeps hub qubits cheap to reach. Its one weakness is that the $f_{\min}$ reservation is uniform across cores, which suits a grid or a ring but not a core graph with a hub (Section V-E).

B. Front Classification

Every front gate is classified under the current layout ℓ. 1Q gates are always directly executable; a CX $g = (q_i, q_j)$ with both operands in core c is *direct-local* if $d_{\mathrm{L}}(c, \ell(q_i), \ell(q_j)) = 1$ and *local-needing-SWAP* otherwise; a CX whose operands lie in different cores is *adjacent-remote* if those cores are core-graph neighbours and *non-adjacent-remote* otherwise. Directly executable gates drain first and completely, intra-core SWAPs are exhausted next, and remote-stage decisions are taken only once no front gate is intra-core. A cost guard caps the front at 96 gates; it binds only on the three largest QPEexact instances, where removing it leaves the emitted stream unchanged, since truncation is by ascending gate index and directly executable gates drain before any routing decision is taken.

C. Intra-Core SWAP Scoring

This stage is SABRE [12]. For a local-needing-SWAP gate set F_{loc}, candidate SWAP edges are the coupling-graph neighbours of every physical position F_{loc} touches, and each is scored under its post-SWAP layout ℓ_a by

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\mathrm{loc}}\, \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\mathrm{loc}} = 0.8, \tag{1}$$

the mean intra-core distance $d_{\mathrm{L}}(c, \ell_a(q_i), \ell_a(q_j))$ over F_{loc}, plus the same mean over an extended set of up to N_{loc}=20 local CX gates from the next L_{loc}=12 DAG layers. The lowest J is applied; ties break on raw front-layer distance, then on a constant-seeded generator. Note the sign convention differs between the two stages: local candidates *minimise* distance, whereas remote plans *maximise* gain (Section III-E).

D. Plan-First Remote Stage

PORTER manipulates remote work as *plans*. A plan is a complete, committable specification of one inter-core action: which primitive fires, which gate or same-control run it resolves, its direction, and consequently its *readiness conditions* — which logical endpoint must sit adjacent to which communication port, and which ports must be free. What it deliberately leaves unspecified is the SWAP sequence that establishes those conditions. Alternatives are compared at the level of *what* remote work to do next, each scored as if its preconditions were met; only the committed plan is handed to the machinery that works out *how*.

Formally, a plan is a tuple

$$\mathsf{p} \;=\; \big(\pi,\; G(\mathsf{p}),\; \mathrm{dir},\; c_{\mathrm{src}},\; c_{\mathrm{dst}},\; \Pi(\mathsf{p}),\; \mathrm{ready}(\mathsf{p}, \ell)\big),$$

with $\pi \in \{\textsc{TeleData}, \textsc{TeleGate}, \textsc{TeleGatePacket}\}$ the primitive, dir its orientation, c_{src} and c_{dst} the cores involved, $\Pi(\mathsf{p})$ the communication ports it reserves, and $\mathrm{ready}(\mathsf{p}, \ell)$ the predicate that holds when p can fire. $G(\mathsf{p})$ is the set of gates the action *discharges*: for a TeleGate, the single front CX it executes; for a TeleGatePacket, the run of same-control gates collected in Section III-F; for a TeleData, the empty set, since moving a qubit executes no gate — its value lies entirely in the gates the relocation makes local, which the scorer estimates (Section III-E).

Three quantities are kept distinct: the plan's *utility*, evaluated under the hypothesis that $\mathrm{ready}(\mathsf{p}, \ell)$ holds whether or not it does; its *readiness cost*, the local work still needed, used only to break ties; and the *preparation policy* that chooses those local SWAPs after p is committed. Two design decisions follow: utility never includes readiness cost, and once a plan is committed PORTER works only on that plan — it generates no other plan and scores no alternative — until the plan fires or is abandoned because no preparation SWAP improves its readiness.

The second is the *commitment rule*, and we adopt it on structural grounds rather than measured ones. A preparation SWAP has no meaning except relative to the plan it prepares, so a router that reopens the choice part-way through preparation can leave the layout carrying moves made towards an action it then abandons; committing makes preparation and the action it serves one unit, and gives the abandonment test of Section III-D something well defined to fire on. It also fixes the granularity at which the plan set is derived — once per committed action rather than once per router iteration. What it does *not* come with is an experiment: no result in Section V isolates either the commitment rule or the plan abstraction beneath it, and Section VI states what that leaves open. Section VII places both against the baselines' choices.

The candidate sets. When the front holds only remote CX gates, PORTER builds two sets. **Normal plans** $\mathcal{P}$ are the actions that could fire once prepared. For each adjacent-remote CX they hold a TeleGate; a TeleGatePacket if a same-control run of ≥ 2 gates can be collected; and, for either endpoint, a TeleData into the partner's core when that core has $f_{\min}$ free positions. A non-adjacent-remote CX cannot be discharged in one action, so it contributes only TeleData hops that move one endpoint to a neighbouring core *strictly closer* to the other endpoint's core, again subject to $f_{\min}$.

Blocked plans $\mathcal{B}$ are the same TeleData candidates whose destination fails the $f_{\min}$ test. PORTER never commits one — executing it would drive a core below the floor. It scores them anyway, as a measurement of what capacity is currently costing: $\mathcal{B}$ names the cores that are saturated and says how much gain is being forgone there, which is what the relief tiers of Section III-G act on.

Selection. Every plan in both sets is scored before any preparation SWAP for it exists; Section III-E gives the score and the ordering key (2) it induces. PORTER then walks $\mathcal{P}$ in that order and commits to the first plan it can actually work on. A plan qualifies if it is *ready* — its endpoints already sit beside the ports it needs and those ports are free, so it can fire immediately — or if it is *preparable*: not ready, but some local SWAP moves the layout strictly closer to readiness. A higher-scoring plan that is neither is skipped rather than waited for, and such plans do occur. A side whose core is completely full is the standard case: no local SWAP can change a core's free-slot count or clear a port that nothing can reach, so no amount of preparation will ever make that plan fire.

Two things can override the choice, and both are driven by the single best-scoring member of $\mathcal{B}$ alone — call it b^*; the rest of $\mathcal{B}$ enters no comparison. If a plan was selected but b^* out-scores it by more than m_{assist}=5.0, PORTER drops the selected plan and relieves the core blocking b^* instead. If nothing in $\mathcal{P}$ was ready or preparable — which includes

$\mathcal{P} = \emptyset$ but is not limited to it — it relieves $\mathcal{B}$'s blocked destinations in ascending hop order. In both cases what is committed is a relief TELEDATA generated for the purpose, never b^* itself. Having committed, PORTER applies only that plan's preparation SWAPs until it is ready, no improving SWAP remains, or the decision state — plan, layout, RNG — repeats, which raises rather than loops.

Preparation SWAPs. A committed plan has exactly two readiness requirements, one per side: source and destination for TELEDATA, control and target for TELEGATE and TELEGATEPACKET, whose target side is routed at execution time (Section III-F). For each unmet requirement, candidate SWAPs are the edges incident to the plan's logical endpoint, or to a nearest free position tied for minimum distance to the relevant port while that port stays occupied. Each is scored by the post-SWAP readiness cost, and at most one improving SWAP per side is applied per pass.

E. Plan Scoring and the Selection Key

Every candidate is scored before any preparation SWAP for it exists, and the scored plans are ordered by one lexicographic key, smallest first:

$$\mathrm{key}(\mathsf{p}) = \big(-\mathrm{total},\ \neg\mathrm{ready},\ \mathrm{prep},\ \mathrm{rank},\ \mathrm{cap},\ -\mathrm{lookahead}\big), \tag{2}$$

each component evaluated at p. The order of the components is the design, so we take them in turn.

total is the estimated gate progress the plan buys, negated so that more progress sorts first. It decides almost every comparison; the five components after it only break ties. Preparation cost is not part of it, so a plan that happens to be cheap to prepare cannot displace one that buys more progress — the property that separates this scorer from DSABRE's, where staging cost is additive (Section VII).

ready is the predicate of Section III-D: the plan can fire under the current layout with no further local work. Ready plans sort ahead of equally valuable unready ones.

prep is the readiness cost — how far the current layout is from satisfying ready, measured as intra-core distance from each endpoint to the port it must reach. It ranks third, below progress and readiness.

rank orders the primitives, preferring TELEDATA (0) to TELEGATEPACKET (1) to TELEGATE (2): when two plans are equal in value, readiness and preparation cost, the one that also improves the layout is kept.

$\mathrm{cap}(\mathsf{p}) = 1/\max(1, f_{\mathrm{dst}} - 1)$ grows as the destination core approaches the capacity floor, so between two otherwise identical plans the one leaving more room wins. It is computed alongside the score and never added to it.

lookahead, negated, is the discounted part of total: among plans tied on everything above, the one whose value reaches further into the circuit is preferred, and a stable plan identifier resolves anything still tied. How far it reaches is set by the horizon L_{tp}, counted in DAG layers. Since a qubit has at most one zero-predecessor gate at a time, at most one of its gates can appear per layer, so a companion cap N_{tp} on the gate count cannot bind whenever $N_{\mathrm{tp}} \geq L_{\mathrm{tp}}$ and L_{tp} alone governs the depth. It is the one parameter PORTER does not fix by constant; Section IV-E1 chooses it per circuit.

That leaves total, which the rest of this subsection computes, taking the three primitives in turn.

Two of them are counts of work the action performs outright. A **TELEGATE** scores 1: the one straddling gate it executes, with nothing left over. A **TELEGATEPACKET** scores the number of straddling gates in its collected run, also with nothing left over, since collection returns a committed executable sequence and every member of it executes once the cat resource is live (Section III-F).

A **TELEDATA** executes no gate at all. Its score has to estimate the gates its moving qubit q will meet once relocated, which needs a measure of how much a move brings a gate closer to being executable.

Gate progress. Write κ for the map sending a logical qubit to the core holding it under the current layout, and κ' for the same map after the move — identical to κ except that q now sits in c_{dst}. For a CX $g = (q_i, q_j)$,

$$\Delta(g) = d_{\mathrm{C}}\big(\kappa(q_i), \kappa(q_j)\big) - d_{\mathrm{C}}\big(\kappa'(q_i), \kappa'(q_j)\big) \tag{3}$$

is the core-graph distance the move removes from g. A TELEDATA moves one qubit one hop, so $\Delta(g) \in \{1, 0, -1\}$: it is 1 whenever the hop brings g's operands one core closer, whether or not that makes them local; 0 for a gate the hop does not touch; and -1 for one it pushes apart.

Three sets of CX gates on q contribute, obtained by simulating the DAG frontier forward under κ':

- $D(\mathsf{p})$, the front CX on q — by the same one-gate-per-qubit fact, the plan's seed gate and nothing else, so $|D(\mathsf{p})| = 1$.
- $L(\mathsf{p})$, the CX gates on q reached by *local closure*. The simulator repeatedly retires every front gate executable under κ' — one-qubit gates, and CX gates whose operands κ' places in one core — which advances the front and exposes gates that were previously blocked behind them. Every CX on q appearing in any front along the way joins $L(\mathsf{p})$, and closure stops at the first front with nothing locally executable. These count as current pressure rather than prediction: reaching them required no gate outside κ''s reach, and the teleport fixes their core distance as soon as they appear.
- $A_k(\mathsf{p})$, the CX gates on q found by continuing past that stopping front, in simulated layer $k \leq L_{\mathrm{tp}}$, discounted by $w_k = \max(w_{\min}, w_0\gamma^{k-1})$ with w_0=0.9, γ=0.9, $w_{\min}$=0.5. These are prediction.

Only here does the score split, into a certain part and a discounted one: $\mathrm{total} = \mathrm{direct} + \mathrm{lookahead}$, where

$$\begin{aligned} \mathrm{direct}(\mathsf{p}) &= \sum_{g \in D(\mathsf{p}) \cup L(\mathsf{p})} \Delta(g), \\ \mathrm{lookahead}(\mathsf{p}) &= \sum_{k \geq 1} \sum_{g \in A_k(\mathsf{p})} w_k\, \Delta(g). \end{aligned} \tag{4}$$

So $L(\mathsf{p})$ enters at full weight alongside $D(\mathsf{p})$, and only $A_k(\mathsf{p})$ is discounted.

F. Same-Control Packet Collection

Same-control CX gates are chained by DAG dependency edges, so they never appear in the front together: a packet

cannot be read off the front and must be collected by forward simulation. From a seed adjacent-remote CX $g = (q_c, q_t)$ with target core c_t, PORTER simulates the DAG frontier forward for up to 120 layers, absorbing any front CX that shares control q_c and has its target in c_t (up to 8 gates), and any front 1Q gate whose qubit is already in c_t, since those execute freely while the cat resource is live. Collection stops at the first unproductive layer or at the size cap. The window counts simulated layers, not collected CXs, so it acts as a cost guard rather than a tuning knob. Because the simulation reaches past the current front, a packet may contain gates a front-limited scan could not. Collection is a heuristic in one respect we have not ablated: stopping at the first unproductive layer can miss a profitable packet separated by harmless intervening work.

Collection only *discovers* the run; nothing is emitted while it simulates. The sequence it returns is recorded on the plan, and if that plan is committed the sequence is emitted in exactly that order — one `cat_entangler`, which consumes the packet's single EPR pair; one `cat_cx` per member, each target first brought beside the port by ordinary local SWAPs that may not touch either live port; and one closing `cat_disentangler`, which frees both ports. Fixing the order at collection time is what lets the scorer count every member as work the packet will actually do: the members are DAG-chained, so executing them in that order respects their dependencies, and no member can become unexecutable once the resource is live.

G. Capacity Relief

Both tiers answer the same question — some core c is at or below $f_{\min}$ and a TELEDATA wants to enter it — by moving a qubit *out* of the congested region rather than into it. Neither picks a destination freely: a breadth-first search from c over the core graph finds the nearest *reservoir*, a core with at least $f_{\min}$ free positions, and the relief action moves one qubit from the penultimate core of that path into the reservoir at the end of it. Every logical qubit in the penultimate core is a candidate, and the candidates are scored and ordered by the plan machinery already described, grouped by hop count so that shorter paths are tried first. Chaining beyond one hop is optional and enabled by default, which lets relief propagate along a path when the immediate neighbour is itself full.

The two tiers differ only in when they fire. **Tail relief** fires reactively, when $\mathcal{P}$ is empty and the front is starved. **Opportunistic relief** fires pre-emptively, while a normal plan is still available, whenever a blocked plan out-scores it by more than m_{assist} — the judgement being that clearing the congestion now is worth more than taking the best currently legal action.

H. Progress and the Failure Model

Deterministic greedy routing on a capacity-constrained machine can stall, and PORTER layers four defences against that. It keeps capacity away from zero: a core with no free physical qubit is terminal, since it cannot vacate an occupied port by local SWAPs, so PORTER legalises the entry layout and admits a TELEDATA into c only while $\text{free}(c) \geq f_{\min}$, which leaves one free position behind. Routing therefore cannot create a terminal core, and a supplied layout containing one is legalised first, by evicting a qubit into the core with the most room; the only input PORTER refuses is one no core has spare capacity to receive such an eviction from. It detects starvation rather than thrashing, an empty $\mathcal{P}$ beside a non-empty $\mathcal{B}$ naming the cores responsible; it manufactures capacity, tail relief turning that signal into work (Section III-G); and it bounds every loop, hashing its decision state once per preparation round and raising if it repeats, and capping the loops with two budgets. Together these make every returned route correct and every run terminating, but they do not guarantee that a run terminates in success; Section VII states the three guarantees precisely and places PORTER against DSABRE, which does carry such a guarantee. A per-iteration cost analysis of the loop is in Appendix I.

IV. Experimental Methodology

A. Research Questions

The evaluation answers four questions, each with its own controls, and Section V is organised around them.

RQ1 **End-to-end cost.** Under each compiler's own recommended protocol, how does PORTER's EPR consumption and compile time compare with TELESABRE's and DSABRE's?

RQ2 **Routing quality with the layout controlled.** Given the *same* externally supplied initial layout, how does PORTER's routing loop compare with DSABRE's — first at an equal route count, then when each is allowed its own refinement schedule?

RQ3 **Mechanism attribution.** What does each of PORTER's mechanisms — capacity relief, the packet primitive, the TELEGATE family, the initial layout and horizon selection — contribute on its own?

RQ4 **Robustness.** How do the answers change with circuit width, core count and core-graph shape?

Each results subsection opens with the question it answers, what is held fixed, and the metric; the interpretation of the answers, including the places where they appear to conflict, is collected in Section VI.

B. Terminology

Budget comparisons depend on using these terms consistently, so we fix them here and use no synonyms.

- A **route** (equivalently a *routing pass*) is one invocation of the routing loop of Section III on one circuit from one layout, producing one operation stream.
- A **candidate layout** is one initial mapping a protocol supplies to the router. A protocol's cost is its layout count times its routes per layout.
- A **protocol** is a named row of Table I: a layout source, a route budget and a selection rule. We avoid "attempt", which the baselines use for both meanings.
- A **candidate horizon** is one value of L_{tp} offered to horizon selection; selecting among them costs one route each.
- **Complete compile cost** is the wall-clock time of an entire protocol — layout generation plus every route over every candidate — not of the winning route alone.

For the machine and the router we use: *core*, *communication port*, and *reservoir* (a core holding at least $f_{\min}$ free positions); a front CX is *local*, *adjacent-remote* or *non-adjacent-remote*; a plan is *normal* or *blocked*, and *ready*, *preparable* or neither. A run that does not complete does so in one of four ways, distinguished throughout: a capacity *deadlock* leaves no legal plan and no relief path, and raises a diagnostic; a *livelock* makes local progress indefinitely while retired gates and EPR count stand still, and is stopped by the iteration guard; a *timeout* exceeds the wall-clock cap; and an *infeasible input* is one the machine has no spare capacity for, refused before routing begins (Section III-H).

C. Setup

Implementation. PORTER is implemented in Python and released with the comparison harness. It takes no seed: heuristic ties resolve through a fixed chain of deterministic keys and any residual tie through a constant-initialised generator, so one route on a fixed circuit and hardware emits a byte-identical stream across runs. Horizon selection is the exception, because its speculative routes share a wall-clock budget: on a loaded machine a candidate may be abandoned that would otherwise have won, so the protocol is reproducible only up to that budget, and the horizon each run selected is recorded with its result. Every reported PORTER route is certified by a replay verifier written separately from the router, which re-derives gate coverage, dependency order and physical legality from the emitted stream alone and is itself tested by fault injection (Appendix D).

Architectures. We use the two grid-of-grids topologies DSABRE [11] evaluates on — architecture A, a 2×2 array of 3×3 cores (36 physical qubits), and architecture H, a 2×3 array of 4×4 cores (96 physical) — adding its 5×5-core scalability series in Section V-D and two networks of IBM 27-qubit heavy-hex tiles in Section V-E. All are read from the baselines' own device files, so every compiler sees identical physical qubit ids.

Benchmarks. We use the same MQT-Bench [9] circuit files DSABRE's evaluation uses, confirmed by exact CX-count match: a 25-qubit suite (AE, GHZ, Graphstate, QFT, QNN, Random) on architecture A, and a 64-qubit suite adding QPEexact, QAOA and Multiplier on architecture H. We add an 80-qubit suite of the same nine families; its AE and QNN instances are not produced by the stock generator and were reconstructed to match its construction, QNN as the deep ZZ-feature-map circuit rather than the 79-CX chain the current generator emits. Appendix A gives both reconstructions and their validation, and the scripts are released.

Baselines. We compare against TELESABRE [10] and DSABRE [11], the latter in its default configuration — safe mode, which makes capacity a legality condition with a per-core reserve and recovers by relaying slack along a BFS path (Section VII).

Three decisions avoid disadvantaging DSABRE. We let its recovery budgets derive from the architecture as its own drivers do, since a binding iteration budget would cut its heuristic search short and hand the remainder to its guaranteed drain at a worse EPR count. We do not charge it for pre-routing layout repair, which lands in its EPR counter although it adjusts the starting layout rather than routing anything, and we net that out for both compilers. And we take its layout rule exactly as it ships rather than substituting one of our own (Section IV-D).

TELESABRE's stock C binary crashes deterministically on architecture A, in its initial-layout pass rather than in routing. All TELESABRE numbers use a minimally patched binary, released with ours, that falls back to the least-loaded core; it changes placement only for qubits the stock binary leaves unplaced, and architecture-H results are unchanged by it. Every compiler invocation is capped at 900 s wall-clock, and a call exceeding it counts as a failure to complete.

D. Initial-Layout Protocols

We evaluate under three initial-layout protocols, designed to separate routing quality from initial-layout quality.

Own-protocol comparison. Each router is first evaluated under its own established best-practice protocol. For the two baselines that is best-of-three, matching the convention already used to compare TELESABRE and DSABRE to each other [11]: TELESABRE runs its built-in Hungarian initial-layout heuristic at config seeds 0–2, and DSABRE runs Qiskit `SabreLayout` on its corners-removed architecture graph at seeds 0–2, each layout routed by SABRE-style forward → backward → forward passes; both report the best EPR count. PORTER instead builds one deterministic temporal locality-greedy layout (Section III-A) and routes it at each of the three candidate lookahead horizons of Section IV-E1, keeping the best — three routing passes over one layout, against nine over three. It searches no layouts, so every number is reproducible from the circuit file alone.

DSABRE's SabreLayout runs on a coupling map with low-degree physical qubits withheld in every core as communication slots, the count fixed by its own fill-adaptive rule. We use that rule as it ships, which is why our DSABRE figures reproduce those it publishes for the architectures it reports.

Shared-layout comparison. To separate layout quality from routing quality, both DSABRE and PORTER are then run from an identical externally supplied mapping — TELESABRE's own initial layout, and separately `pytket-dqc`'s hypergraph partition (Appendix B) — so the only difference left is the routing loop. **Cross-layout comparison.** Finally each compiler routes the other's own layout, which closes the remaining cell of the design.

E. Operating Protocols

Section III defines one routing pass. Three protocols run that pass more than once and keep the best result by EPR; because they invoke the router repeatedly rather than changing what a single pass does, we describe them here rather than as part of the algorithm. All three are deterministic given the circuit, the hardware and the configuration, and none takes a seed.

1) Lookahead-horizon selection: The teleport-lookahead horizon L_{tp} is the one parameter of Section III-E for which no tested constant is best across the corpus. In an offline sweep over $\{4, 8, \ldots, 1024\}$ on every instance the best value

spans the whole range: 80-qubit QNN wants 4, the 25-qubit suite wants 16, and every 100-, 200- and 360-qubit instance wants 128, beyond which EPR saturates. The gap between PORTER's configured default of L_{tp}=16 and the per-instance best is 17.2% geometric-mean EPR.

Interaction density sets the direction. For each logical qubit that appears in at least one CX, count the distinct partners it shares a CX with anywhere in the circuit; ρ is the mean of that count over those qubits, divided by $n-1$. It is the mean fraction of the other logical qubits each qubit ever interacts with: $\rho \to 1$ is all-to-all, every qubit meeting essentially every other, while a linear CX chain gives $\rho \approx 2/(n-1)$. The two regimes behave differently. When ρ is near 1 a long horizon finds the moving qubit wanted in every core at once, so the gain estimate degenerates into noise and short horizons win; when ρ is small the signal is coherent and long horizons win. Across the benchmark ρ ranks the best horizon at Spearman -0.765, but it is not a usable predictor alone: in the band $0.42 \le \rho \le 0.53$, where most 64-qubit circuits sit, QAOA wants 8 and AE wants 128, and a threshold rule on ρ recovers only 8.0% of the available 17.2%.

PORTER therefore uses ρ to *prune* rather than to choose. A circuit with $\rho \ge 0.80$ is routed at $L_{\mathrm{tp}} \in \{4, 8, 16\}$, every other circuit at $\{8, 16, 128\}$, and the lowest-EPR route is kept. No instance above the threshold ever selected the long horizon, so pruning costs no EPR and removes the dominant compile-time term: a long horizon that fails to converge and consumes the whole protocol's budget. Two rules bound the cost. The circuit's configured horizon is routed first, unguarded, and wins ties. The remaining candidates share a wall-clock budget proportional to what that first route cost, and one that overruns it is abandoned. The guarantee this gives is conditional and worth stating exactly: *provided the configured-horizon route completes*, selection returns no higher an EPR count than that route would alone. It can still raise compile time, and which speculative candidates finish — hence which route is selected — depends on the wall-clock budget and so on the machine (Section IV-C).

This rule was fitted on the corpus it is evaluated on. The sweep, both candidate sets and the 0.80 threshold were fixed by inspecting the 48 benchmark instances, and there is no held-out family, so evidence for the *size* of the gain is in-sample. Two things bound the risk. The rule is a portfolio, not a prediction: the configured horizon always competes and wins ties, so a poor candidate set costs compile time but cannot worsen the selected route. And the direction the sweep exposes — that no tested constant is best across circuits whose densities differ by an order of magnitude — does not depend on the values chosen. An out-of-sample validation remains to be done.

*2) Backward refinement (*PORTER+*R):* DSABRE routes each candidate layout with a forward → backward → forward schedule, so a protocol that matches its routing effort rather than only its layout count is needed to separate the two effects. PORTER+R supplies one. The schedule is SABRE's bidirectional layout refinement [12] applied to whole routes. Let $R(C, \ell)$ denote routing C from ℓ and ℓ' the layout it ends on. Pass 1 computes $R(C, \ell_0)$; pass 2 computes $R(C^{\mathrm{rev}}, \ell'_1)$ on the reversed circuit and *discards its route*, keeping only ℓ'_2; pass 3 computes $R(C, \ell'_2)$. Reversing C reverses every dependency edge, so routing it from the forward pass's end state yields a layout adapted to the *start* of the forward circuit. Odd pass counts above three repeat the cycle; P=5 throughout. Because the best forward pass is selected, PORTER+R can never report a higher EPR count than the pass 1 route it contains — at the cost of P routes where one would do. It adds no randomness.

PORTER+R refines from the *selected* route: pass 1 is horizon selection, and the horizon it chooses is held for the remaining $P-1$ passes. Choosing once rather than per pass keeps the pass count linear — re-selecting every pass would multiply it by the number of candidates to re-decide a property of the circuit that refinement does not change — and starting from the selected route is what makes PORTER+R contain PORTER, so the best-pass rule bounds it below. Total routes are $|\mathcal{H}| + P - 1 = 7$. The cost is wall-clock rather than passes: a deep selected horizon makes a later pass dear, and on one instance a pass is stopped by the per-pass guard instead of converging.

One property of the schedule matters for reading its results. Refinement consumes a post-routing layout, and routing destroys the per-core free-slot reservation ℓ_0 was built with (Section III-A), so an intermediate pass may start from a layout worse than ℓ_0 for PORTER's own capacity machinery. Individual passes therefore regress on some instances, and only the best-pass rule keeps that from surfacing.

*3) Layout-discount search (*PORTER+*S):* PORTER+S searches the one free scalar in the layout heuristic — the per-layer discount γ of Section III-A, fixed at 0.99 elsewhere — in two phases.

Phase 1 builds a layout for each $\gamma \in \{0.80, 0.90, 0.95, 0.99, 1.0\}$ and routes it once at the configured horizon — five routes — keeping the lowest-EPR layout. Phase 2 runs the P=5 refinement schedule above from two seeds, phase 1's winner and the γ=0.99 layout when that is not already the winner, adding $P-1$=4 routes each; refining the phase-1 winner alone is not reliably best, so both are run. The total is $5 + 4 = 9$ routes when γ=0.99 wins and $5 + 4 + 4 = 13$ otherwise, plus the $|\mathcal{H}| - 1$ extra probes each seed's selection spends: 10 to 17 routes, the range Table I reports. Since γ=0.99 always competes and is refined from its selected route, PORTER+S cannot return a higher EPR count than PORTER does.

We previously also reported PORTER+SR, the better of horizon selection and the γ-search. It is now redundant: phase 2 refines from the γ=0.99 layout among its seeds, which is the layout PORTER builds by default, so the search contains the selected route. Section V-F confirms the two never differ.

F. Comparison Protocols and Budgets

Table I states, for every configuration that appears in the results, where its initial layouts come from, how many routing passes it spends, and what it selects on. We use the names in its first column throughout.

Two notions of a matched budget are in play and neither is universally fair, so we report against both. Equal *route counts*

TABLE I

EVERY CONFIGURATION THAT APPEARS IN SECTION V: LAYOUT SOURCE, LAYOUT COUNT $|\mathcal{L}|$, ROUTING PASSES PER LAYOUT P, TOTAL ROUTES, AND THE QUANTITY MINIMISED OVER THEM. ROUTES ARE $|\mathcal{L}|\cdot P$ EXCEPT WHERE A SCHEDULE SELECTS A HORIZON, WHICH ADDS $|\mathcal{H}|-1$ PROBES PER SEED, AND FOR PORTER+S, WHICH SPENDS ONE PASS ON EVERY CANDIDATE LAYOUT AND THE FULL REFINEMENT SCHEDULE ON ONE OR TWO OF THEM. LOWER EPR IS BETTER EVERYWHERE.

Name	Compiler	Layout source	$\|\mathcal{L}\|$	P	routes	selects on
Own protocol (RQ1, RQ4)						
TELESABRE	TELESABRE	Hungarian, seeds 0–2	3	1	3	best EPR
DSABRE	DSABRE	SabreLayout, seeds 0–2	3	3	9	best EPR
PORTER	PORTER	temporal locality-greedy	1	3	3	best EPR
PORTER+R	PORTER	temporal locality-greedy	1	5	7	best forward pass
PORTER+S	PORTER	$\gamma \in \{0.80, 0.90, 0.95, 0.99, 1.0\}$	5	1 or 5	10–17	best EPR
Shared layout (RQ2) — both compilers routed from one external layout						
DSABRE shared	DSABRE	TELESABRE's or pytket-dqc's	1	3	3	best pass
PORTER shared	PORTER	the identical layout	1	3	3	best EPR
PORTER+R shared	PORTER	the identical layout	1	5	7	best forward pass
Cross layout (RQ2) — each compiler on the other's own layout						
DSABRE on PORTER	DSABRE	PORTER's layout	1	3	3	best pass
PORTER on DSABRE	PORTER	DSABRE's candidates	3	3	9	best EPR
Ablation (RQ3) — one fixed layout, one fixed horizon						
ablation legs	PORTER	temporal locality-greedy	1	1	1	—

TABLE II

MAIN COMPARISON, OWN PROTOCOL: EACH COMPILER FROM ITS OWN INITIAL LAYOUT AND ROUTE BUDGET (TABLE I). PORTER+R IS PORTER WITH BACKWARD REFINEMENT. EPR COLUMNS ARE GEOMETRIC MEANS OVER THE CIRCUITS *all four* COMPILERS COMPLETE, SO THEY EXCLUDE THE SEVEN TELESABRE DOES NOT CONVERGE ON; "DONE" COUNTS THE CIRCUITS EACH COMPILER COMPLETES, OUT OF THE SUITE SIZE. t IS COMPLETE PROTOCOL TIME (GEOMETRIC MEAN, SECONDS) OVER THE WHOLE SUITE. LOWER IS BETTER THROUGHOUT. PER-CIRCUIT EPR AND SWAP COUNTS ARE IN APPENDIX H.

	TELESABRE		DSABRE		PORTER		PORTER+R	
Suite	EPR	t	EPR	t	EPR	t	EPR	t
25q (6)	106.6	0.51	45.8	0.23	43.1	**0.11**	**38.8**	0.26
64q (9)	284.5	11.89	**144.6**	**1.77**	166.2	1.98	156.1	4.00
80q (9)	121.0	5.11	44.9	**3.57**	38.5	**3.95**	**36.6**	14.04
done	17 / 24		24 / 24		24 / 24		24 / 24	

give each compiler the same number of router invocations from one identical layout — three each, in the shared-layout rows. Equal *candidate-layout sets* instead hand PORTER the very layouts DSABRE generated for itself, so both choose their best from the same three starting points; that equalises the search over layouts rather than the routing effort spent on each. Wall-clock is not equalised anywhere, but every table reports complete compile time so a reader can renormalise. PORTER costs more per route than DSABRE (Section V-F), so equal route counts flatter it on time and not on effort.

G. Metrics and Aggregation

The primary metric is EPR pairs consumed: TELESABRE counts teledata plus telegate operations, DSABRE counts teledata, and PORTER counts teleports plus telegates plus telegate packets, one pair per packet, all three the same physical resource. Intra-core SWAP counts and compile time are secondary and are never traded against EPR by any selection rule. Aggregates are unweighted geometric means over circuits, so each circuit counts equally; percentage changes are $100(b/a-1)$ against the stated baseline and ratios are b/a, both computed from those means rather than averaged per circuit. Failures are never imputed: a configuration that does not complete an instance is excluded from every mean it appears in, along with that instance's other columns where the comparison is paired, so a mean labelled (n) is over exactly the n circuits all its columns completed — which flatters a baseline that fails its hardest instances, and is why completion counts accompany every such mean. Compile times are wall-clock on one otherwise idle machine (Apple M2, macOS 26.5, CPython 3.13), measured once per configuration with no warm-up, so they are indicative and should be read as ratios between protocols rather than as absolute costs.

V. RESULTS

Each subsection answers one research question of Section IV-A. Protocol names and budgets are those of Table I; lower EPR is better throughout.

A. RQ1: End-to-End Cost Under Each Compiler's Own Protocol

Question: how do the three compilers compare when each uses its own recommended protocol? *Held fixed:* nothing — each compiler supplies its own layouts and pass schedule (Table I). *Metric:* geometric-mean EPR over jointly completed circuits, plus completion counts and complete compile time.

Table II summarises the three suites; Appendix H gives the per-circuit EPR and SWAP counts. Baseline counts come from each baseline's own harness; on the eight 64-qubit circuits TELESABRE completes, ours reproduces the counts DSABRE publishes for it [11] exactly.

Against TELESABRE the ordering is consistent across the suite: from one deterministic layout PORTER lowers geometric-mean EPR by 59.6%, 41.6% and 68.2% at 25, 64 and 80 qubits, spending three routes against TELESABRE's three seeded runs, and every circuit TELESABRE completes improves, from -16.0% to -81.0%. Refinement adds to this at 25 and 64 qubits (63.6% and 45.1%) and little at 80, where the selected route already accounts for most of it (68.2% against 69.8%) — the suite where the horizon, not the extra routes, does the work. PORTER and DSABRE complete all twenty-four instances; TELESABRE completes seventeen, failing on 64q Random and six of the nine 80-qubit circuits.

Against DSABRE the answer varies by suite. Routing its three candidate lookahead horizons and keeping the lowest-EPR result (Section IV-E1), against DSABRE's three candidate layouts at three passes each, PORTER takes the geometric mean at 25 qubits (43.1 against 45.8) and at 80 (38.5 against 44.9) and trails at 64 (166.2 against 144.6; all three on the TELESABRE-matched basis), winning ten instances, losing twelve and tying two; pooled over all twenty-four it is within 2%, 174.9 against 171.9. Refinement takes the lead pooled (163.0 against 171.9) at seven routes to DSABRE's nine, and the γ-search extends it to 159.1 (Section V-F). PORTER's largest saving is 80q Multiplier, 2,132 against 4,841 with refinement; its worst loss is 64q QFT (262 against 179).

TABLE III
LAYOUT AGAINST ROUTER: GEOMETRIC-MEAN EPR OVER ALL TWENTY-FOUR INSTANCES FOR EVERY COMBINATION OF INITIAL LAYOUT AND ROUTING LOOP. ROWS ARE THE THREE LAYOUT SOURCES, COLUMNS THE ROUTERS; PORTER SPENDS THREE ROUTES PER LAYOUT AND DSABRE THREE, WHILE PORTER+R SPENDS FIVE. EVERY CELL IS OVER THE FULL SUITE — NO INSTANCE IS EXCLUDED, BECAUSE ALL THREE LAYOUT SOURCES PLACE ALL TWENTY-FOUR CIRCUITS. LOWER IS BETTER; THE BEST ENTRY IN EACH COLUMN IS BOLD.

Initial layout	DSABRE	PORTER	PORTER+R
DSABRE's `SabreLayout`	171.9	175.2	158.5
PORTER's temporal locality-greedy	171.2	174.9	163.0
`pytket-dqc`'s partition	**164.1**	**166.4**	**155.2**

B. RQ2: Routing Quality With the Layout Controlled

Question: with the initial layout held identical, does either routing loop use fewer EPR pairs? *Held fixed:* the layout, taken in turn from each of the three sources available. *Metric:* geometric-mean EPR over all twenty-four instances.

The three sources in Table III are those that place all twenty-four circuits. As TELESABRE converges only on seventeen instances (Section V-A) and emits an initial layout only where it converges, we therefore don't include it in the comparison.

The routing loops are equivalent within noise; the layouts are not. Read across any row of Table III: from the same starting layout at three routes each, the two land within 2% of one another everywhere, DSABRE ahead in all three rows and winning twelve of the twenty-four instances to eight on the `pytket-dqc` row. This is the controlled test and PORTER loses it. Read down the columns, though, and `pytket-dqc`'s hypergraph partition is the best start for *both* routers — 4.5% better than DSABRE's own layout is for DSABRE, 4.8% better than PORTER's own is for PORTER. The spread between layout sources exceeds the spread between routing loops, so the initial layout is the weaker component of both compilers rather than an advantage to either.

Refinement pays from every starting layout, once the horizon is free to move. PORTER+R gains 9.5% over PORTER on DSABRE's layout (175.2 → 158.5) and 6.8% on both the others (174.9 → 163.0, 166.4 → 155.2), overtaking DSABRE's own protocol from all three at seven routes to its nine. An earlier version of this schedule, which held the lookahead horizon at its configured value, gained on DSABRE's layout and nothing at all on `pytket-dqc`'s — which invited the reading that refinement merely repairs a poor starting layout. That reading was an artefact of the fixed horizon: refining from the selected route instead (Section IV-E2) recovers as much from the best layout source as from the worst.

C. RQ3: Mechanism Ablation

Question: what does each mechanism contribute on its own? *Held fixed:* the initial layout, the lookahead horizon and every other parameter; exactly one mechanism is disabled per column. *Metric:* EPR and completion.

Table IV disables each mechanism in isolation, holding the initial layout and every other parameter fixed; Appendix G tabulates the per-action counts behind these rows. The six

TABLE IV
MECHANISM ABLATION: PORTER EPR WITH ONE DESIGN DECISION DISABLED AT A TIME, ALL FROM THE SAME TEMPORAL LOCALITY-GREEDY LAYOUT. COLUMNS: "NO PK" DISABLES PACKETS (`MAX_PACKET_SIZE`=1), "NO REL." DISABLES BOTH RELIEF TIERS, "TELEDATA" REMOVES THE TELEGATE FAMILY; "PK" AND "TG" ARE THE PACKET AND SINGLE-TELEGATE COUNTS OF THE FULL CONFIGURATION. EACH GMEAN ROW IS OVER THE INSTANCES *all its filled columns* COMPLETE, AND ONLY THOSE COLUMNS ARE FILLED: n=14 PAIRS FULL AGAINST NO PK AND NO REL., n=16 PAIRS FULL AGAINST TELEDATA. ROWS ARE THEREFORE COMPARABLE ACROSS A ROW AND NOT DOWN A COLUMN. †FAILS TO COMPLETE (DEADLOCK OR ITERATION-GUARD TRIP). ‡COMPLETES ONLY WITH THE ADDITIONAL RELIEF TIER DESCRIBED IN THE TEXT, WHICH THE TELEGATE OTHERWISE MAKES UNNECESSARY. THE FIRST GMEAN ROW IS OVER THE FOURTEEN INSTANCES THE FIRST THREE CONFIGURATIONS SHARE; THE SECOND IS THE TELEDATA-ONLY COLUMN PAIRED AGAINST FULL OVER THE SIXTEEN INSTANCES IT COMPLETES.

Suite	Circuit	Full	No pk	No rel.	TELEDATA	pk	tg
25q	ae	65	71	80	68‡	8	0
25q	qft	74	74	140	74	0	0
25q	qnn	119	122	212	119	0	0
25q	random	401	412	395	413‡	43	93
64q	ae	177	211	327	198‡	32	0
64q	qft	262	372	382	**216**‡	45	1
64q	qnn	864	931	600	**694**‡	240	5
64q	random	737	843	789	†	77	228
80q	ae	490	688	†	**354**‡	264	15
80q	qft	446	435	†	**307**‡	209	2
80q	qnn	1517	1571	†	**926**‡	448	30
80q	random	11040	11628	†	†	572	3538
gmean, paired n=14		57.6	61.4	67.0			
gmean, paired n=16		78.3	83.9		**71.7**		

omitted rows — GHZ and Graphstate at every scale — form no packets, never trigger relief and issue no telegates, so all configurations coincide there.

Packets. Disabling TELEGATEPACKET makes every shared-control remote CX run pay per gate, and the cost tracks the packet count: +42% EPR on 64q QFT (262 → 372, 45 packets), +40% on 80q AE (490 → 688, 264 packets), against a +7% overall gmean; the lone counter-move (80q QFT, 446 → 435) shows packet formation is a heuristic commitment that can displace a slightly better teleport plan. Read alone this column is a clean win for the primitive; the next overturns that reading.

TELEDATA only. This column is two configurations. *Strict removal* — deleting the TELEGATE family and changing nothing else — is the one-factor ablation, and it fails: ten of the twelve rows do not complete. The TELEGATE is the only remote action that needs no free *data* slot in the destination core. Like every remote action it needs the two inter-core communication qubits free, but the cat copy it creates lives on the target-side communication qubit rather than displacing a logical qubit, so unlike TELEDATA it is not gated on $\mathrm{free}(c) \geq f_{\min}$ and is always available for an adjacent-remote CX. Without it, a front of core-adjacent remote CXs whose destinations all sit at or below $f_{\min}$ has an empty plan set, and the tail-relief BFS is offered only non-adjacent candidates. The TELEGATE is thus essential to *completion* here under the unchanged relief policy — not to termination, which the

guards of Section III-H provide either way.

The vocabulary variant, marked ‡, additionally feeds adjacent-blocked candidates to that BFS, which the full configuration never needs; it recovers all but the two Random rows, the most telegate-heavy in the suite. Adding the same tier to the full configuration leaves every row EPR-identical, so what follows is a difference of vocabulary and not of relief — but not a strict ablation. On that basis a teleport-only vocabulary is the cheaper one, increasingly so with scale (+2% at 25q, mixed at 64q, −33% at 80q; 8.5% below Full over the sixteen instances it completes), and the circuits it wins are the packet-heaviest, giving TELEDATA only < Full < No packets: packets beat the single telegates they degrade into, but pure teleportation beats both. Section VI argues this is a currency mismatch in the plan comparator rather than a verdict on packets. It also answers a question DSABRE leaves open: its teledata-only vocabulary was chosen by measurement, with grouping untested. Grouping does close the per-EPR gap, and still does not justify spending pairs that teleporting would have spent better.

Capacity relief. Disabling both tiers costs completion, not just EPR pairs: 80q AE, QFT, QNN and Random all fail, because with every destination below $f_{\min}$ and no way to drain qubits toward reservoirs the loop has no legal remote plan left. Where routing does complete, dense circuits pay heavily (+89% on 25q QFT, +78% on 25q QNN, +85% on 64q AE), though two are cheaper *without* it — 64q QNN (864 → 600) and marginally 25q Random — so the opportunistic tier can overpay where congestion would have cleared itself. With both tiers on no instance in this evaluation reached the deadlock path of Section III-H; with both off four did. That is a statement about these instances, not a guarantee. The initial layout, which this table holds fixed, is assessed in Sections V-B and V-E.

D. RQ4: Scale-Up on QFT and QPEexact to 360 Logical Qubits

Question: do the RQ1 and RQ2 answers hold as circuit width and core count grow together? *Held fixed:* DSABRE's own scalability architectures and circuit files. *Metric:* geometric-mean EPR and complete compile time.

Every result so far is at 108 physical qubits or fewer. Table V takes the two circuit families that exist at 100, 200 and 360 logical qubits onto DSABRE's scalability series: 5×5 cores throughout, core count 6 → 12 → 20, so occupancy stays roughly constant while the core graph grows and inter-core traffic with it — DSABRE's own cost rises 200 → 765 → 1,890 on QFT.

PORTER uses fewer EPR pairs than DSABRE on all six instances, by a widening margin: 0.90× at 100 logical qubits, 0.67× at 200, 0.62× at 360, for 0.72× overall. It does so at 0.54× DSABRE's complete compile cost (8.4 s against 15.4 s), routing three times where DSABRE routes nine.

The horizon, not the layout, accounts for the reversal. The intervention that closes PORTER's gap at the smaller sizes does not work here: handing PORTER DSABRE's own `SabreLayout` candidates — the control that reaches parity at 25 and 64 qubits and wins on both heavy-hex core graphs — makes it *worse* (0.96× against 0.72× in our released results), worst on the largest instance, where DSABRE's layout costs 1,791 pairs against the 1,156 PORTER's own layout reaches. The horizon does: every instance in this series selects L_{tp}=128, and at the configured 16 PORTER trails DSABRE by 1.45× (Section IV-E1). Among the mechanisms we varied, that is the one that accounts for the change.

This separates two deficits the smaller suites conflate. At 36 to 108 physical qubits the initial layout limits PORTER: control it, match the route count, and the difference largely disappears. At 150 and above it does not, and the horizon does. Selecting it per circuit is a workaround, not a fix; a K-aware horizon remains open.

TABLE V
SCALE-UP SUITE: QFT AND QPEEXACT AT 100, 200, AND 360 LOGICAL QUBITS, EPR PAIRS. EACH COMPILER USES ITS OWN INITIAL LAYOUT AND ITS OWN PROTOCOL. ARCHITECTURES FOLLOW DSABRE'S OWN SCALABILITY PROTOCOL — 5×5 CORES THROUGHOUT, CORE COUNT 6 → 12 → 20 — READ FROM THE BASELINES' OWN DEVICE FILES SO ALL ROUTERS SEE IDENTICAL PHYSICAL QUBIT IDS; OUR DSABRE COLUMN REPRODUCES THE FIGURES IT PUBLISHES FOR EXACTLY THIS SERIES [11] (201/246/766/896/1890/1938) TO WITHIN THE ONE CANDIDATE LAYOUT OUR PROTOCOL ADDS. "t" IS COMPLETE PROTOCOL TIME IN SECONDS — LAYOUT GENERATION PLUS EVERY ROUTE OVER EVERY CANDIDATE, THE METRIC USED THROUGHOUT: NINE ROUTES FOR DSABRE, THREE FOR PORTER. `PYTKET-DQC` IS OMITTED: ITS HYPERGRAPH PARTITIONER DOES NOT FINISH IN REASONABLE TIME AT THESE WIDTHS. LOWER IS BETTER IN EVERY COLUMN.

		architecture			DSABRE		PORTER	
Circuit	CX	n	P	K	EPR	t	EPR	t
QFT	3,420	100	150	6	200	3.6	**198**	2.8
QPEexact	3,579	100	150	6	246	4.3	**200**	2.2
QFT	7,220	200	300	12	765	13.9	**579**	9.5
QPEexact	7,579	200	300	12	896	13.9	**528**	8.9
QFT	13,300	360	500	20	1890	53.9	**1156**	19.5
QPEexact	13,979	360	500	20	1938	83.1	**1224**	33.1
gmean (6)					705.7	15.4	**507.7**	**8.4**

E. RQ4: Heavy-Hex Cores and Non-Grid Core Graphs

Question: how much of the foregoing depends on square cores wired in a grid? *Held fixed:* the 64-qubit circuit suite and the physical-qubit count; the core shape and core graph change. *Metric:* geometric-mean EPR and win/loss counts.

To test how much of PORTER's behaviour depends on square cores wired in a grid we adopt DSABRE's non-grid experiment [11]: the same nine-circuit 64-qubit suite on four IBM 27-qubit heavy-hex tiles. Two things change at once — the *core* becomes irregular (degrees over $\{1, 2, 3\}$, six degree-1 leaves, diameter 12 against a 4×4 grid's 6) and the *core graph* is either a ring, every core of degree 2, or a star with core 0 a hub. PORTER's hardware object is derived from the architecture DSABRE's own builder produces, and a topology assertion checks that both compilers see identical qubit ids, edges and port endpoints before anything is routed. Neither compiler is specialised for a heavy-hex tile.

TABLE VI
THE NINE-CIRCUIT 64-QUBIT SUITE ON FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL QUBITS), WIRED AS A *ring* (4 LINKS, EVERY CORE DEGREE 2) AND A *star* (3 LINKS, CORE 0 A HUB EVERY ROUTE MUST CROSS). GEOMETRIC-MEAN EPR, EACH COMPILER UNDER ITS OWN INITIAL LAYOUT AND OWN PROTOCOL; PORTER COMPLETES ALL EIGHTEEN INSTANCES, SO EVERY ROW IS OVER THE FULL NINE. PER-CIRCUIT ROWS, AND THE FIXED-LAYOUT CONTROL IN WHICH PORTER ROUTES DSABRE'S OWN SABRELAYOUT CANDIDATES, ARE IN APPENDIX H. DSABRE REPRODUCES ITS PUBLISHED NUMBERS FOR THESE TWO ARCHITECTURES ON ALL EIGHTEEN INSTANCES [11].

Core graph	DSABRE	PORTER
ring (9)	**85.2**	86.1
star (9)	**101.1**	122.3
star Multiplier	722	**455**

The core graph matters more than the core shape. On the ring (Table VI) the two compilers are within 1% of one another, 86.1 against 85.2: doubling the intra-core diameter and replacing degree-4 interiors with degree-1 leaves — a tile neither compiler was designed for — disturbs neither. The star does. There PORTER trails by 21%, 122.3 against 101.1, winning two of nine. The core shape is the same in both rows; only the core graph differs.

Star Multiplier is a livelock the horizon escapes. At a fixed L_{tp}=16 this instance does not terminate. It fails as a local-SWAP livelock rather than the capacity deadlock of Section III-H — retired gates freeze at 3,226 of 13,040 and the EPR count at 538 while intra-core SWAPs accumulate one per iteration, so the iteration guard stops it — and the failure is configuration-dependent: the same circuit, architecture, layout and router complete at L_{tp}=128, at 455 pairs against DSABRE's 722, a 37% reduction and PORTER's widest margin anywhere on these two architectures. Since horizon selection routes that candidate as a matter of course, PORTER has no unrouted instance anywhere in this evaluation.

The initial layout is the dominant observed source of the star gap. Routed instead from DSABRE's own `SabreLayout` candidates (Appendix H), PORTER overtakes DSABRE on *both* core graphs — 82.9 against 85.2 on the ring, 98.5 against 101.1 on the star — and the reversal is larger on the topology it loses under its own layout. The cause is in the layout: PORTER reserves $f_{\min}$ in every core, treating cores as interchangeable, which they are on a grid and a ring and are not on a star, where Multiplier opens with free counts $(2, 2, 13, 27)$ — the hub every route must cross pinned at the floor while a whole core sits empty. Raising the reservation uniformly does not repair it (we tried $f_{\min}\in\{4, 6\}$); a reservation scaled by a core's position in the core graph is untried.

Taken with Section V-D, this bounds the layout reading rather than extending it: up to 108 physical qubits, across two core shapes and three core graphs, the initial layout separates the two compilers and controlling it erases the difference; above 150 the horizon takes over.

TABLE VII
PROTOCOL TRADE-OFF OVER ALL 24 INSTANCES. "ROUTES" IS THE NUMBER OF ROUTING PASSES; TIMES ARE COMPLETE COMPILE COSTS (GEOMETRIC MEAN PER INSTANCE). PORTER ROUTES EACH OF THREE CANDIDATE LOOKAHEAD HORIZONS AND KEEPS THE BEST (SECTION IV-E1); PORTER+R REFINES BACKWARD FROM THE SELECTED ROUTE (SECTION IV-E2); PORTER+S ADDS THE BEST-OF-γ LAYOUT SEARCH ON TOP OF THAT. PORTER+SR IS NOT LISTED; THE TEXT SAYS WHY.

Protocol	routes	EPR	vs. DSABRE	time (s)	vs. DSABRE
DSABRE	9	171.9	1.00	1.38	1.00
PORTER	3	174.9	1.02	1.25	**0.91**
PORTER+R	7	163.0	0.95	3.22	2.34
PORTER+S	10–17	**159.1**	**0.93**	9.01	6.55

F. The EPR/Time Operating Frontier

Table VII places PORTER's operating points (Section IV-E) against DSABRE, pooled over all twenty-four instances. Each is the previous one plus a search: PORTER selects the horizon, PORTER+R refines backward from the route it selects, and PORTER+S searches the layout discount as well. Because every schedule contains the one before it and keeps the best result by EPR, the column is monotone by construction, and none of them can report more EPR than routing once at the configured horizon.

The trade is time against EPR pairs. PORTER costs 1.02× DSABRE's EPR at 0.91× its compile time; refinement takes the lead at 0.95× the EPR for 2.34× the time, on seven routes against DSABRE's nine; the layout search reaches 0.93× at 6.55×. Increasing PORTER's deterministic search budget therefore closes and reverses the own-protocol gap. We stop short of calling that gap a search-budget gap: the added schedules change the initial layout as well as the route, and the comparison equalises route counts rather than wall-clock time — which is what PORTER actually spends, since a deep selected horizon makes each pass dearer rather than adding passes.

PORTER+SR, which took the better of horizon selection and the γ-search, is no longer a distinct operating point. The search refines from the γ=0.99 layout among others, and that is the layout PORTER builds by default, so with refinement running on top of selection (Section IV-E2) the search contains the selected route and cannot lose to it. It does not differ from PORTER+S on any of the twenty-four instances, at 19% more compile time, so we drop it. The same argument makes each of the three listed schedules safe against the one below it, which is why none of them needed to be reported with a caveat about losing to a cheaper configuration: at a fixed horizon they did, and refining from the selected route is what removed that.

All times are *complete* compile costs — layout generation plus every route over every candidate, not the winning route alone — and PORTER's advantage there is in the count of routes, not their price. It is the dearer of the two per route on all nine 64-qubit circuits, taking 3.6× as long in geometric mean: enumerating whole plans, collecting packets by forward simulation and running a two-tier capacity search all cost more per iteration than scoring a flat list of hops, and a deep horizon costs more still. On that subset DSABRE is cheaper end to end

(1.77 s against 1.98 s); pooled over all twenty-four PORTER is slightly ahead at $0.91\times$, and at $1.02\times$ DSABRE's EPR the cheap operating point remains PORTER's.

VI. Discussion and Limitations

Several results in Section V read as tensions when placed side by side. We resolve them here, then state what the evaluation does not establish.

The remote stage is not isolated by any experiment. The *plan-oriented* design — whole inter-core actions as candidates, ranked on gain with preparation cost demoted to a tie-break — is the defining structural difference in PORTER's remote stage, and *commitment* is the narrower rule that a selected plan is not reopened until ready. Neither is ablated here: every column of Table IV varies the action vocabulary or the capacity policy, and none replaces the plan with a hop. We therefore do not attribute the reductions of Section V-A to either. Those are produced by the whole system — layout, relief, vocabulary, scoring, horizon and protocol together — and isolating the abstraction's share would need a hop-granular candidate set, which is a different router. This is the clearest gap in our attribution.

Layout binds below about a hundred physical qubits, the horizon above it. PORTER trails DSABRE under RQ2 at ≤ 108 physical qubits, and controlling the layout at a matched budget erases the difference. On the scale-up series neither that control nor extra refinement helps — DSABRE's layout is worse for PORTER there than its own — while a longer horizon moves PORTER from $1.45\times$ behind to $0.72\times$ ahead. Both regimes are consistent with the horizon being an absolute gate count that does not grow with the core count K, so a plan at twenty cores is scored over proportionally less of the circuit than at four. We have not tested a K-scaled horizon, so this is a hypothesis, not a finding.

Why TeleData-only can beat the full action set. Packets beat the single telegates they degrade into, yet removing gate teleportation entirely is cheaper still on the packet-heaviest 80-qubit circuits. The cause is a currency mismatch in the scorer. A packet's gain counts remote CX gates resolved immediately; a TeleData plan's is a sum of core-distance reductions, most of it discounted over a bounded horizon. Nothing establishes that one resolved gate and one unit of distance reduction are commensurate, and where a teleport repays past the horizon the comparator cannot see it. The ablation is evidence that this mismatch costs EPR pairs at scale; it is not evidence that a pair spread over a run of gates is a bad idea.

What the completion record does and does not show. Section VII separates correctness and termination, which PORTER guarantees by construction, from completion, which it does not. Every instance in this evaluation completes under the selected-horizon protocol, but that is a property of the protocol and the benchmark rather than of the router: at a fixed horizon, heavy-hex star Multiplier livelocks, and the extra candidate is a way of not meeting the failure rather than of excluding it. Only one of the four failure modes of Section IV-B was observed for PORTER here — livelock at a fixed horizon on heavy-hex star Multiplier — and capacity deadlock appeared only once relief was disabled. DSABRE's safe mode carries a termination-with-success proof under stated architectural hypotheses, and supplying a comparable guarantee is the clearest piece of theory this design leaves open.

A. Threats to Validity

Benchmark coverage. The main suites are nine MQT-Bench families at three widths; the scale-up series is two families (QFT and QPEexact) only, so nothing here supports a claim about scaling behaviour on irregular or randomly structured circuits at those widths. The two heavy-hex networks are two core graphs, not a survey of architectures.

Parameter rules fitted in-sample. The horizon candidate sets and the density threshold were chosen by inspecting these same instances (Section IV-E1). The magnitude of the horizon-selection gain is therefore in-sample. The portfolio construction bounds the downside — the configured horizon is always a candidate and wins ties — but an out-of-sample validation has not been performed. The same caution applies in weaker form to $f_{\min}$, m_{assist} and the packet window, which were fixed early and not swept.

Reconstructed benchmark instances. The 80-qubit AE and QNN circuits are not produced by the stock generator and were reconstructed (Appendix A). We release the construction scripts and the files, and validate the QNN reconstruction by exact CNOT-sequence match against the 64-qubit file, but they remain our circuits rather than the generator's.

Patched baseline binary. All TeleSABRE numbers use a binary patched to avoid an out-of-range index in its initial-layout pass (Section IV-C). The patch changes placement only for qubits the stock binary leaves unplaced, and architecture-H results, where the stock binary does not crash, are unchanged by it; we have not otherwise audited its effect.

Unequal native budgets. The own-protocol comparison gives DSABRE nine routes against PORTER's three. This is each compiler's own convention rather than a controlled budget, which is why RQ2 exists and why compile time is reported everywhere.

Aggregation over failures. Geometric means are taken over jointly completed instances, so a baseline that fails hard instances is scored only on easier ones. Completion counts are reported alongside every such mean, but the two must be read together.

Implementation and timing. PORTER is Python and the baselines are Python and C respectively, so per-pass times compare implementations as much as algorithms. Compile times are wall-clock on one machine. Horizon selection additionally guards its speculative routes with a wall-clock budget, so on a loaded machine it may abandon a candidate that would otherwise have won: the protocol's output is reproducible only up to that budget, though each individual route is exactly reproducible.

Cost model. One EPR pair per inter-core action, independent of hop count and generation protocol; link contention, entanglement latency and fidelity, purification, and classical communication cost are all outside the model (Section II-B). Conclusions are relative to that model.

VII. Related Work

TeleSABRE [10] is the published state of the art for this cost model: it supports teledata and gate teleportation and builds on SABRE's extended-set scoring. The two differ chiefly in decision structure. TeleSABRE scores inter-core candidates through a contracted-communication-graph Dijkstra energy that folds staging cost and lookahead into one value, and — uniquely among the compilers here — places those candidates in the *same* pool as local SWAPs, taking the global minimum each iteration. PORTER keeps local work strictly ahead of inter-core work, scores whole plans independently of preparation cost, generates preparation SWAPs only for the plan it commits to, and adds the packet primitive. Its capacity handling is also more proactive: alongside TeleSABRE's safety-valve cap it maintains an explicit blocked-plan set with a tail-relief search and a pre-emptive relief comparator.

dSABRE [11] attacks the same problem, and the two compilers are close relatives. We run it in its default configuration throughout, and our numbers match those it publishes for the architectures it reports.

Stating what the two share keeps either from being credited with the other's ideas: both rank moves by SABRE's front-plus-lookahead Δ score; both are strictly locality-first, scoring an inter-core candidate only once no front gate is intra-core; both promote capacity from a scoring penalty to a legality condition with the same floor of two free slots; and both manufacture capacity by relaying a free slot from a reservoir along a BFS path rather than evicting arbitrarily. Four differences remain, and they are what this paper evaluates: PORTER's action vocabulary is larger (dSABRE ships teledata only); its unit of commitment is a whole plan rather than one hop; preparation cost enters as a tiebreaker rather than additively; and its initial layout is one deterministic construction rather than a best-of-N search.

The fifth difference is not a design choice but a gap, and it runs dSABRE's way. Three guarantees are worth separating here. *Correctness*: a returned operation stream realises every gate of the input circuit and violates no coupling or port rule. *Termination*: every run ends, in success or by raising a diagnostic or tripping a guard. *Completion*: that a run ends in success. PORTER provides the first two by construction (Section III-H) and the third only empirically. dSABRE's safe mode provides all three: it carries a termination-with-success proof under stated architectural hypotheses — a connected core graph, connected intra-core couplings, and enough spare physical capacity per core. PORTER has no analogue, and supplying one is the clearest piece of theory this design leaves open. What we can say is measured rather than proved, and Section V reports it: no instance in this evaluation is left unrouted.

Table VIII places the four systems on the axes that matter here.

Other approaches. Partition-first compilers assign qubits to cores by hypergraph or community partitioning [13]–[15] and route the residue; `pytket-dqc` [14] is the mature representative, compared against in Appendix B under the caveat that its e-bit unit assumes long-lived entanglement and different communication memory. AutoComm [8] identifies gate sequences that can share one entangled resource, which PORTER's packet applies inside a routing loop rather than as a pre-pass (Section II-C). SABRE [12] supplies the scoring skeleton all three compilers inherit.

TABLE VIII
Design axes of the compilers discussed here. "Commitment" is the unit a scoring decision is taken over. `pytket-dqc` is partition-first and does not route with these primitives, so several axes do not apply.

	TeleSABRE	dSABRE	pytket-dqc	PORTER
Paradigm	online	online	partition-first	online
Primitives	data, gate	data	(e-bit)	data, gate, packet
Local/remote	one pool	staged	—	staged
Commitment	one move	one hop	whole plan	whole plan
Prep. cost	in score	in score	—	tie-break
Layout	Hungarian, 3 seeds	SabreLayout, 3 seeds	hypergraph partition	one deterministic
Capacity	cap	legality + relay	partition	legality + relief
Termination	—	with-success proof	—	correct, terminates
Metric	EPR	EPR	e-bits	EPR

VIII. Conclusion

PORTER is a deterministic DQC compiler built on a plan-oriented remote stage: it scores a complete inter-core action on the gate progress it buys, before any preparation SWAP for it exists, and commits before preparing. Around that it places a capacity discipline with two relief tiers, a deterministic initial layout, and a per-circuit choice of teleport-lookahead horizon.

Three conclusions are supported. First, on the evaluated MQT-Bench suites PORTER uses substantially fewer EPR pairs than published TeleSABRE — 59.6%, 41.6% and 68.2% less in geometric mean at 25, 64 and 80 logical qubits, at three routes each — and completes every instance in all three suites, including the seven on which TeleSABRE does not converge. Second, against dSABRE, PORTER is competitive at the sizes both target — $1.02\times$ its pooled EPR at $0.91\times$ its compile time under each compiler's own protocol — and its advantage grows with the machine, reaching $0.72\times$ its EPR pairs on the 100–360-qubit scale-up series. At the smaller sizes which compiler leads depends on the initial layout and the routing budget, and Section V-B reports both directions. Third, the ablations attribute these results rather than leaving them to the system as a whole: capacity relief is what lets a deterministic-greedy design complete in practice; the initial layout dominates the difference between the two compilers at the smaller sizes evaluated, and the lookahead horizon at the larger ones.

One ablation result is negative and it delimits the contribution. A richer gate-teleportation vocabulary does not always pay: at 80 qubits, removing gate teleportation entirely beats the full action set by 32.8% on the packet-heaviest circuits, which we read as the plan scorer undervaluing teleportation's latent return rather than as a verdict on amortisation.

The evaluation is bounded: two circuit families at the largest widths, two non-grid core graphs, horizon rules fitted on the corpus they are measured on, and a cost model charging one pair per inter-core action that ignores link contention,

latency and fidelity (Section VI-A). Five openings follow: a completion guarantee to match DSABRE's, an out-of-sample validation of the horizon rule, a horizon that scales with core count rather than being chosen by trial, a comparator that weighs a resolved gate against a latent distance reduction on one scale, and a hardware cost model under which EPR count is not the whole objective.

ACKNOWLEDGMENTS

This work was partially supported by the Natural Science Foundation of China (Grant No. 12471437). The authors used Anthropic's Claude (via Claude Code) to assist with: (i) reviewing and applying behaviour-preserving performance fixes to the PORTER implementation, (ii) authoring the independent replay verifier and the cross-router comparison harness, (iii) diagnosing and patching the TELESABRE reference-implementation issue described in Section IV-C, and (iv) drafting the LaTeX source of this manuscript. All AI-generated content was reviewed and verified by the authors, who take full responsibility for the correctness and originality of the work.

REFERENCES

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[4] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[7] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

[8] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[9] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[10] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[11] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[12] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[13] P. Escofet, M. Bandic, C. G. Almudever, and S. Feld, "HQA: A hungarian-based qubit assignment for multi-core quantum computers," in *Proceedings of the IEEE Quantum Software Conference*, 2023.

[14] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[15] O. Daei, K. Navi, and M. Zomorodi-Moghadam, "Optimized quantum circuit partitioning," in *International Journal of Theoretical Physics*, vol. 59, 2020, pp. 3804–3820.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.